Testing

12: Programmable DMA

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

Announcements

- Research Desktop is back!
- P3 dealine pushed back to Monday
- Hardware Demo

DMA: Direct Memory Access

- A mini-CPU that does copy for you:

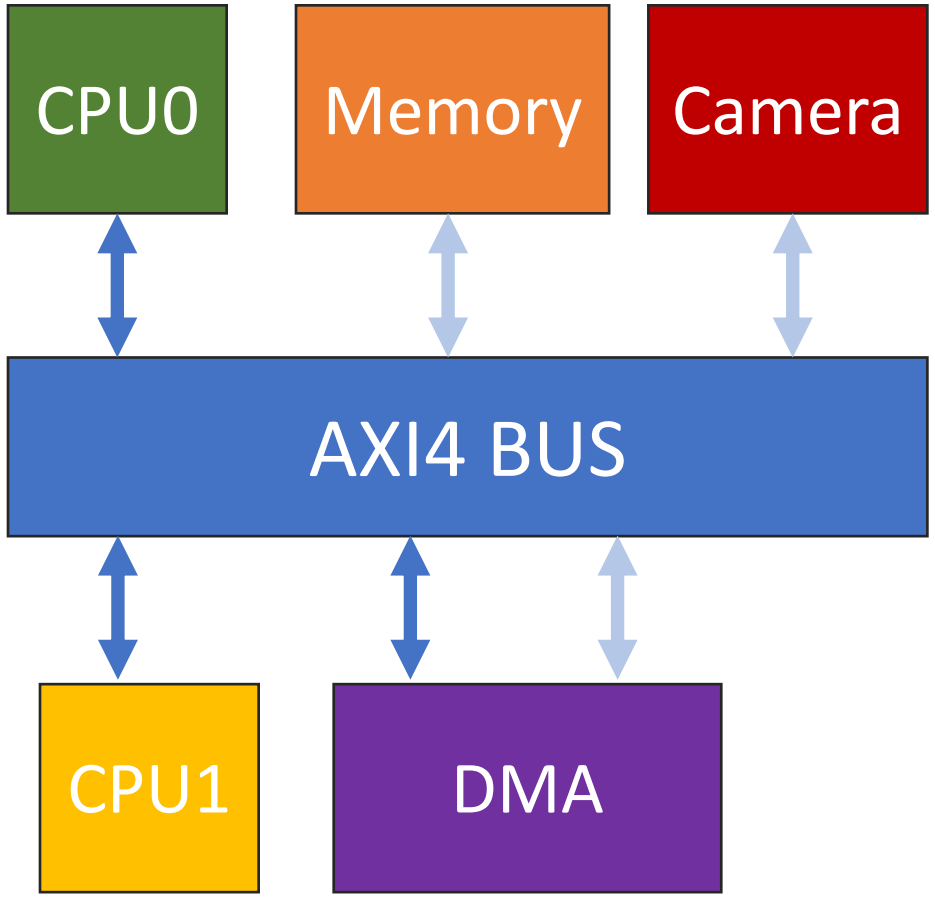
```
void copy (uint32_t * from,
               uint32_t * to,
               uint32_t size)
{
   register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = *from;

        to[i] = reg;
    }
}
```

DMA has 2 Memory Interfaces



- Interface 1: Memory Copy
  - Data Interface
  - Fast
  - Master
- Interface 2: Tell DMA <u>what</u> to copy
  - Control Interface
  - Slower
  - Slave

DMA V1.0 Interface

- 0x0400: Control Register (Start)
- 0x0404: Status Register (Done)
- 0x0408: Source Address
- 0x040C: Destination Address
- 0x0410: Transfer Size in Bytes

Using DMA from the CPU:

0x0400: Control Register
0x0404: Status Register
0x0408: Source Address
0x040C: Destination Address
0x0410: Transfer Size in Bytes

```
void dma_copy ( uint32_t * from,
                uint32_t * to,
                uint32_t size){

    *((uint32_t*)(0x0408))=from;
    *((uint32_t*)(0x040C))=to;
    *((uint32_t*)(0x0410))=size;
    *((uint32_t*)(0x0400))= 0x1; //start
    //spin until copy done?
    while( *((uint32_t*)(0x0404)) != 0x1){;}
}
```

`volatile` keyword omitted!

Other DMA tweaks

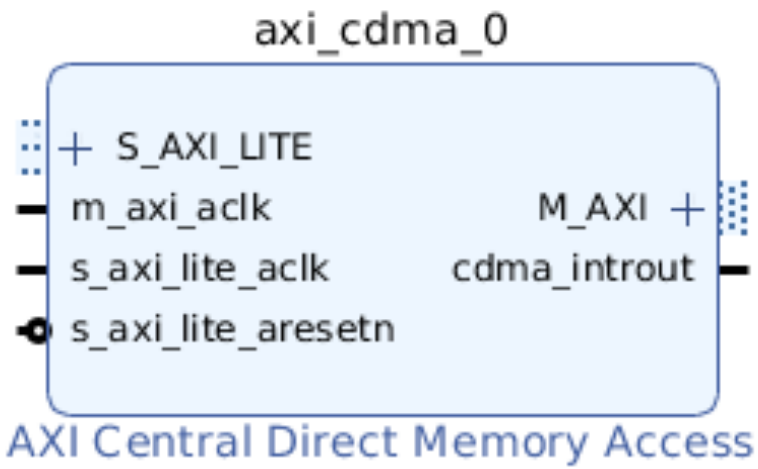
```
void dma_copy (uint32_t * from,
               uint32_t * to,
               uint32_t size,
               uint32_t inc_from, uint32_t inc_to)
{
   register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = (inc_from ? *from[i] : *from);

        if (inc_to) to[i] = reg;
        else        to = reg;
    }
}
```

DMA in Xilinx



Xilinx terminology:

- Central Direct Memory Access : CMDA
  - Standard DMA
- Direct Memory Access (DMA)
  - Programmable DMA!

Real DMA

Register Address Map

Table 2-6: **AXI CDMA Register Summary**

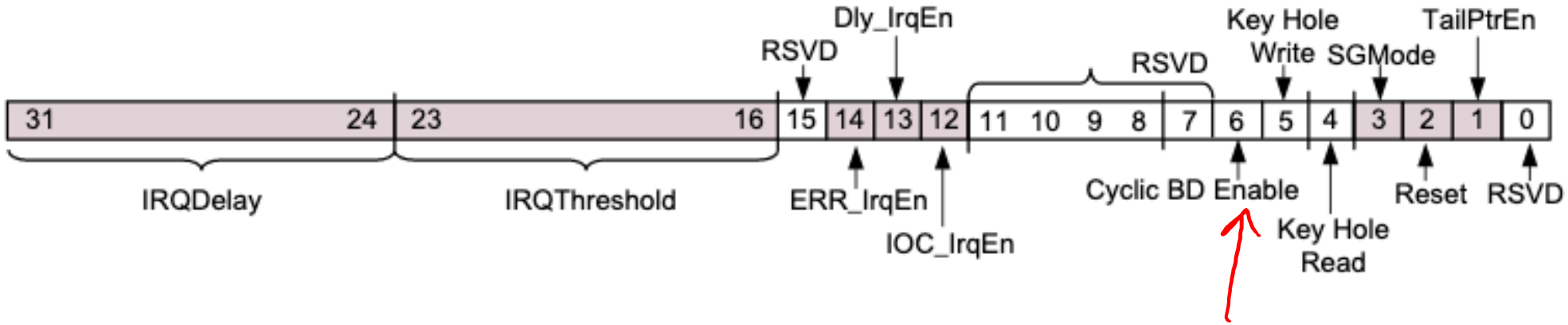
Address Space Offset[1]	Name	Description
00h	CDMACR	CDMA Control
04h	CDMASR	CDMA Status
08h	CURDESC_PNTR	Current Descriptor Pointer
0Ch[2]	CURDESC_PNTR_MSB	Current Descriptor Pointer. MSB 32 bits. Applicable only when the address space is greater than 32.
10h	TAILDESC_PNTR	Tail Descriptor Pointer
14h[2]	TAILDESC_PNTR_MSB	Tail Descriptor Pointer. MSB 32 bits. Applicable only when the address space is greater than 32.
18h	SA	Source Address
1Ch[2]	SA_MSB	Source Address. MSB 32 bits. Applicable only when the address space is greater than 32.
20h	DA	Destination Address
24h[2]	DA_MSB	Destination Address. MSB 32 bits. Applicable only when the address space is greater than 32.
28h	BTT	Bytes to Transfer

Real DMA

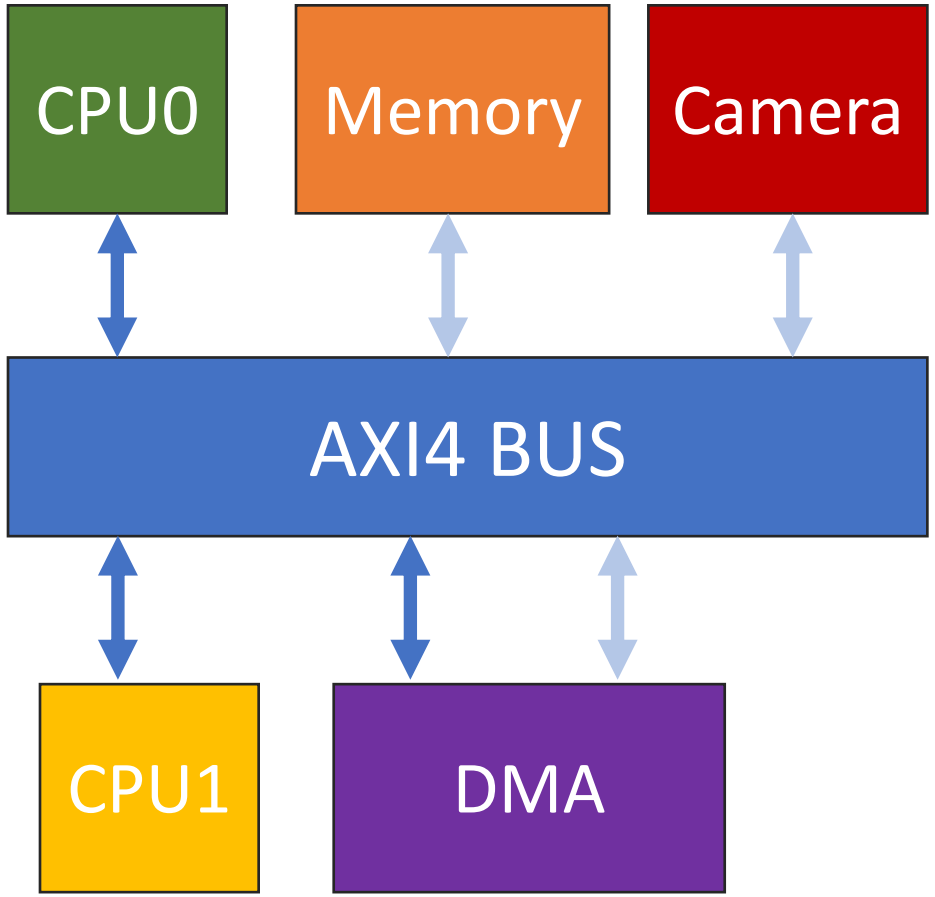
Register Details

CDMACR (CDMA Control – Offset 00h)

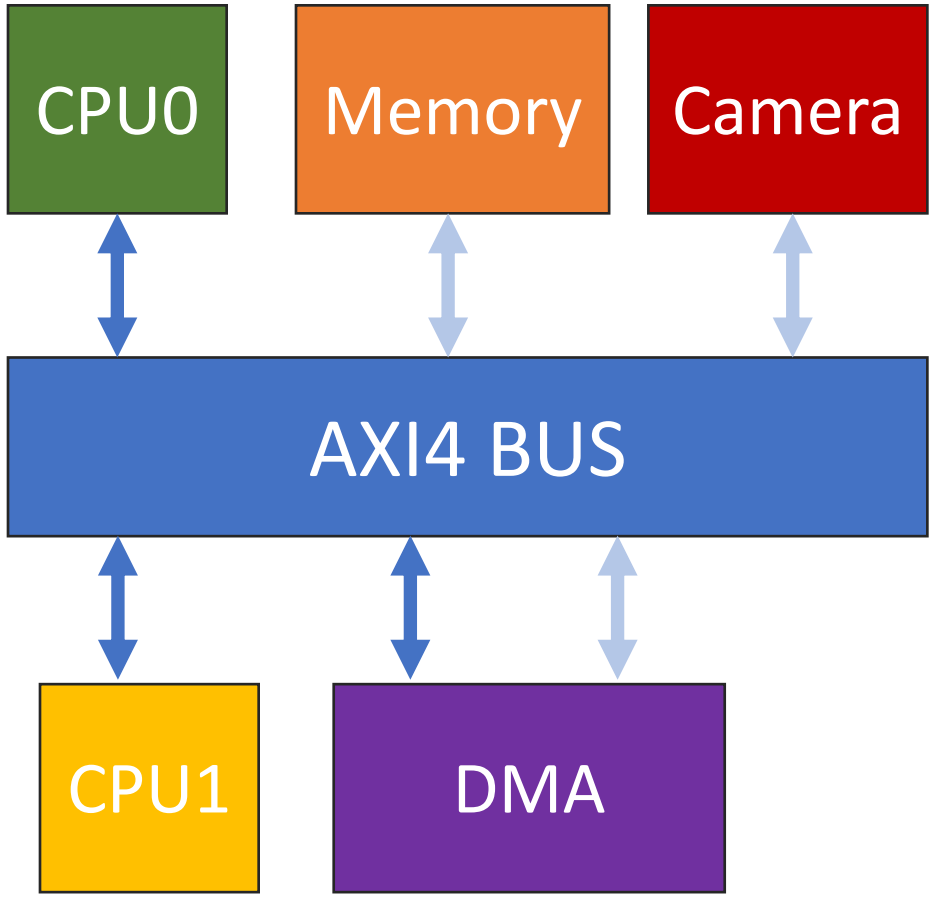
This register provides software application control of the AXI CDMA.



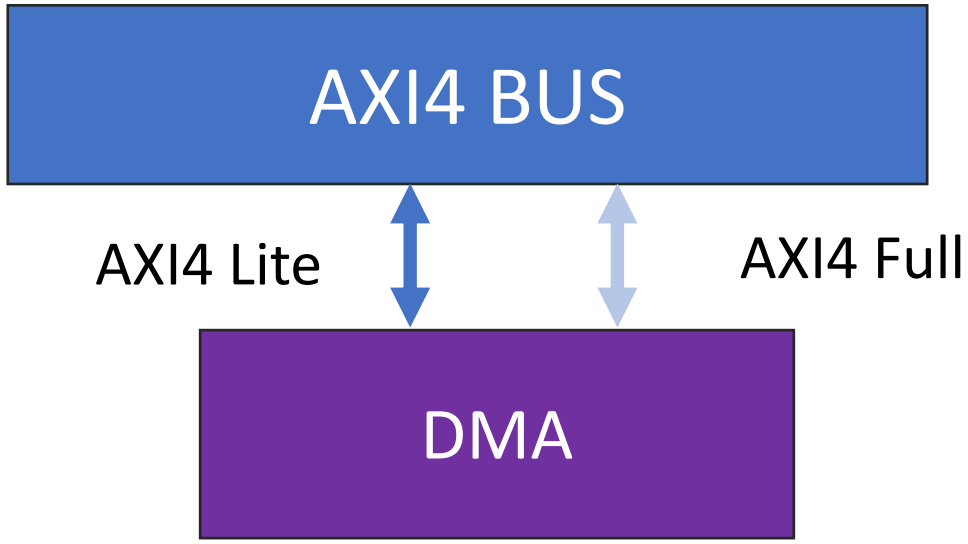
DMAs copy by load + store



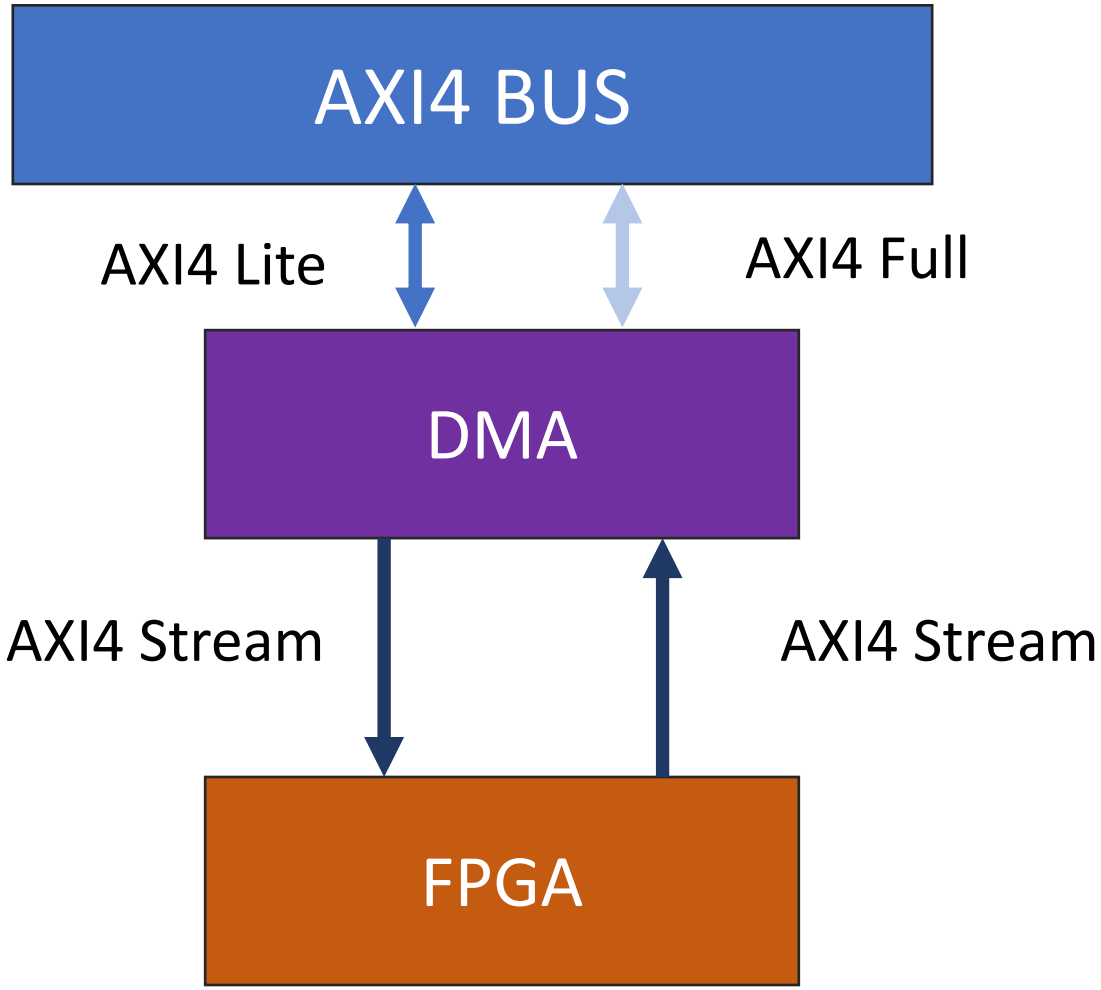
With FPGAs, DMAs can copy with load + **compute** + store!



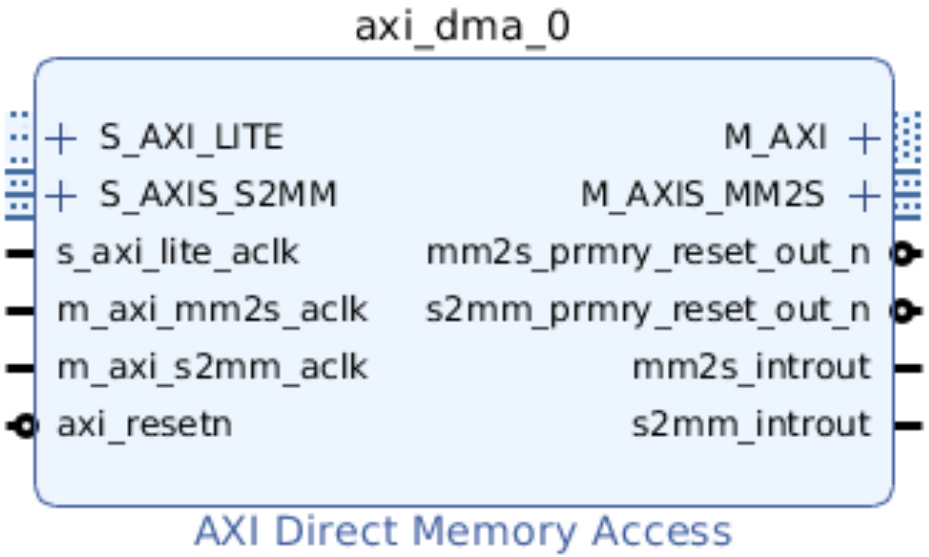
Xilinx DMAs allow you to **program** what the DMA does



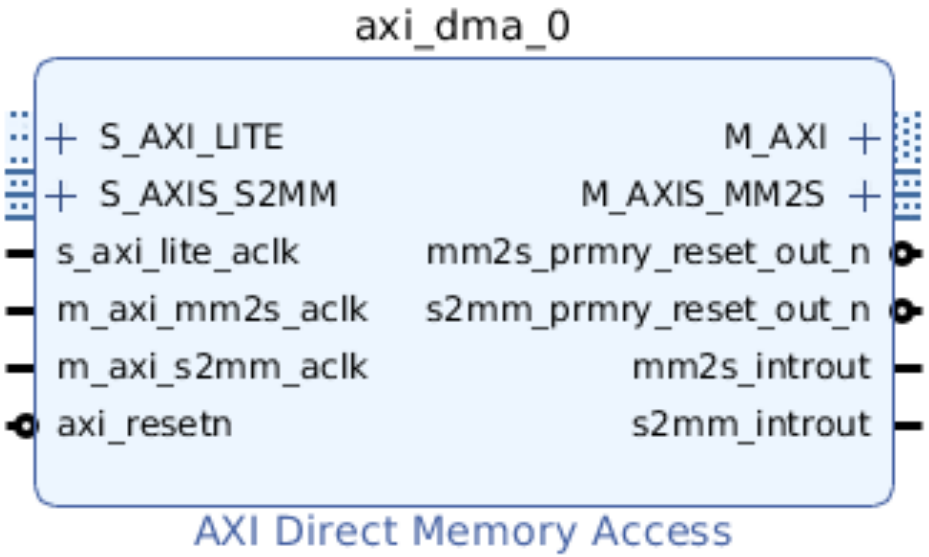
Xilinx DMAs allow you to **program** what the DMA does



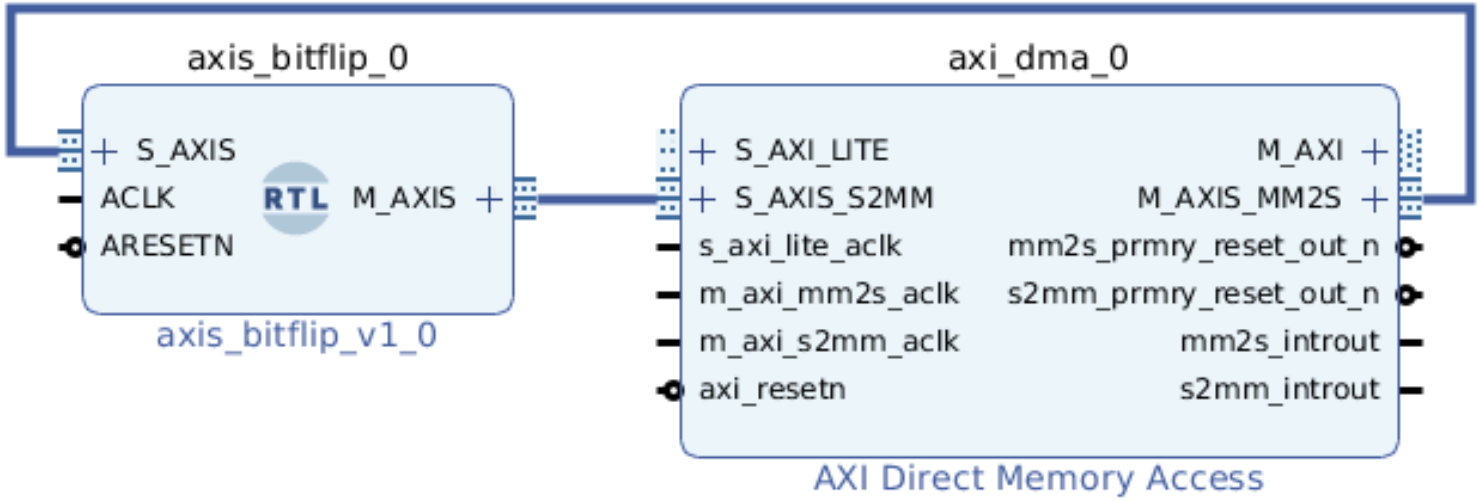
Xilinx Programmable DMA



Can we make this into a regular DMA?



DMA that flips all the bits in the copy



```
module axis_bitflip(
    input           ACLK,
    input           ARESETN,
    input [31:0]    S_AXIS_TDATA,
    input [3:0]     S_AXIS_TKEEP,
    input           S_AXIS_TLAST,
    input           S_AXIS_TVALID,
    output          S_AXIS_TREADY,
    output [31:0]   M_AXIS_TDATA,
    output [3:0]    M_AXIS_TKEEP,
    output          M_AXIS_TLAST,
    output          M_AXIS_TVALID,
    input           M_AXIS_TREADY
    );
```

```
    assign M_AXIS_TDATA= ~S_AXIS_TDATA;

    assign M_AXIS_TKEEP= S_AXIS_TKEEP;
    assign M_AXIS_TLAST = S_AXIS_TLAST;
    assign M_AXIS_TVALID= S_AXIS_TVALID;
    assign S_AXIS_TREADY= M_AXIS_TREADY;

endmodule
```

Bitflip in C

- INPUT = { 0x0, 0x1, 0x2, 0x3}
- DMA (w/bitflip)
- OUTPUT = { 0xf, 0xe, 0xd, 0xc}

Next Time

- Memory Protection

12: Programmable DMA

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University